

Sampling Coding Practice Set II

Sampling — Coding Practice Set II

Signals and Linear Systems · 25 stand-alone problems · each sized for 25-30 minutes

How to use this set

Each of the twenty-five problems below is a **self-contained exercise of roughly half an hour**, made of two to four functions that build on one another. Pick any one problem and finish it; they do not depend on each other except that a few reuse `folded_frequency` from Problem 1.

The starter file `sampling_practice.py` contains every function with its body replaced by `raise NotImplementedError`. The harness at the bottom of that file is provided — **do not modify it**. Running

```
python sampling_practice.py
```

prints one line per problem: `PASS` when every function in that problem is correct, `FAIL` or `ERROR` otherwise. A whole finished set prints `25 of 25 problems passed`.

Conventions used throughout:

- `f`, `fs` are ordinary frequencies in Hz; `w` (ω) is angular, in rad/s.
- `T = 1/fs` is the sampling interval and `ws = 2*pi/T` the sampling frequency in rad/s.
- Sample times are always `t_n = n/fs` for `n = 0, 1, ..., N-1`, with `N = int(duration*fs)` where a duration is given.
- `numpy` is the only import you need (Problem 5 also uses `fractions.Fraction`).
- `np.sinc(x)` is the **normalised** sinc, $\sin(\pi x)/(\pi x)$. Mind the normalisation.
- Anything the docstring says about shapes, ordering or ranges is part of the contract and is tested.

Problem 1 — Aliasing analyser (fold, partner, identify)

Build the three pieces of a tool that answers “what will this tone look like after sampling, and can I read it back?”

folded_frequency(f, fs) — the apparent baseband frequency of $\cos(2\pi f t)$ sampled at `fs`, as a float in $[0, fs/2]$. Must be correct for **any** `f` ≥ 0 , including `f > fs/2`, exact multiples of `fs`, and exact multiples of `fs/2`. The answer is *not* `f % fs`: frequencies in the upper half of each band fold back downwards, because a real cosine cannot tell `+f` from `-f`.

alias_with_phase(f, phi, fs) — assume $0 < f < fs/2$. Return `(f_alias, phi_alias)` where `f_alias` is the smallest positive frequency other than `f` whose samples coincide with those of $\cos(2\pi f t + \phi)$, and `phi_alias` is the phase that makes them coincide, reported in $[0, 2\pi)$. The reflected branch runs the angle backwards, so the phase does **not** survive unchanged — derive what happens to it.

identify_tone(x, fs) — given `N` samples of a single cosine $A \cos(2\pi f n/fs + \phi)$ whose frequency sits exactly on the DFT grid, recover `(A, f, phi)` from the real FFT. Watch three things: the amplitude scaling of a real cosine’s bin, the bin-to-hertz conversion (which uses the time-record length, not the `rfft` length), and the dc bin, which must not be allowed to win the `argmax`. Report `phi` in $[0, 2\pi)$.

Tested with: the three folds above; a phase match to machine precision over `n = 0 ... 49` for `f = 300`, `phi = 0.7`, `fs = 1000`; and recovery of `(1.0, 125.0, 0.4)` from 256 samples at `fs = 1000`.

Problem 2 — Multi-tone sampling simulator and rate selection

sample_signal(components, fs, duration) — `components` is a list of `(A, f, phi)` triples. Return `(t, x)` for the sum of those cosines sampled on the standard grid, with `N = int(duration*fs)`.

apparent_components(components, fs) — return, in the input order, one `(A, f_app, phi_app)` triple per component describing what survives sampling. Amplitude is untouched; the frequency folds as in Problem 1; and if the component lands on the reflected branch the phase must be negated too. Report each phase in $[0, 2\pi)$.

`min_rate_without_alias(components, candidate_rates)` — return the smallest rate from `candidate_rates` at which the signal is recoverable, or `None` if none is. Two conditions must both hold: every component sits strictly below $f_s/2$, and no two components land on the same apparent frequency (two tones summing onto one baseband line cannot be separated afterwards, even if neither individually folded). Guard the distinctness test against floating-point dust.

Tested with: a 50-sample record at $f_s = 1000$; the component $(1.0, 700, 0.5)$ becoming $(1.0, 300, -0.5 \bmod 2\pi)$ while $(1.0, 100, 0.25)$ passes through untouched; and `min_rate_without_alias` choosing 700 out of $[400, 700, 900]$ for components at 100 and 300 Hz.

Problem 3 — Three reconstruction filters, measured against the truth

`sinc_reconstruct(samples, fs, t)` — Whittaker-Shannon interpolation, $x_r(t) = \sum_n x[n] \text{sinc}((t - nT)/T)$, evaluated at every time in the array `t`. Build the full $\text{len}(t) \times \text{len}(\text{samples})$ weight matrix by broadcasting and finish with a single matrix-vector product; a nested Python loop will be uncomfortably slow. Remember `np.sinc` is already normalised.

`zoh_reconstruct(samples, fs, t)` — the DAC staircase: the output at `t` is sample `floor(t*fs)`, clamped so times before the record give `x[0]` and times at or past the end give `x[N-1]`.

`linear_reconstruct(samples, fs, t)` — first-order hold, i.e. convolution with the triangular `h1(t)`, which is exactly what `np.interp` does pointwise (including the endpoint clamping). The only trap is the sample-time grid.

`compare_reconstructions(f, fs, duration, t)` — sample a unit cosine, rebuild it three ways, and return `{"sinc": ..., "linear": ..., "zoh": ...}` holding the maximum absolute error against the exact cosine at the query times.

Before you run it, predict the ordering, then check that your numbers agree with the frequency-domain picture: the ideal brick wall, then $T \text{sinc}^2$ with its fast rolloff, then $T \text{sinc}$ with the worst image leakage. Explain to yourself why the sinc error is not exactly zero.

Tested with: $f = 3$, $f_s = 40$, $\text{duration} = 5$, `t` spanning $[2, 3]$; requires `sinc < 0.01`, the strict ordering `sinc < linear < zoh`, and `zoh > 0.3`.

Problem 4 — The hold filters in the frequency domain

`hold_magnitude(w, T, order)` — $|H_0(jw)| = T |\text{sinc}(wT/2\pi)|$ for `order = 0` and $|H_1(jw)| = T \text{sinc}(wT/2\pi)^2$ for `order = 1`. Accept a scalar or a numpy array and return the same shape. Say in one line why the $\exp(-jwT/2)$ factor never appears, and why the first-order response is the square of the zeroth-order one.

`droop_db(order)` — the in-band attenuation at the band edge $w = w_s/2$ relative to dc, in dB. It is scale-free, so evaluate at $T = 1$. You should recover the familiar ZOH figure near -3.92 dB and exactly twice that many dB for the FOH.

`image_leakage_db(order, T)` — the second complaint about the holds is that their gain outside the passband is not zero, so spectral images leak into the analogue output. Quantify it: integrate $|H(jw)|^2$ over $[0, w_s/2]$ and over $[w_s/2, 2.5w_s]$ using the trapezoid rule on the fixed grid `np.linspace(0, 2.5*ws, 100001)`, and return $10 \log_{10}(\text{image}/\text{baseband})$.

`zoh_compensation_gain(f, fs)` — the digital pre-emphasis that cancels the droop, $1/\text{sinc}(f/fs)$. Show for yourself that $wT/(2\pi)$ is exactly f/fs , so the gain is 1 at dc and $\pi/2$ at the band edge. Accept scalars or arrays.

Tested with: both droop figures in closed form; `hold_magnitude(0, 2, 0) == 2`; `image_leakage_db(0, 1) ≈ -6.20` dB and `image_leakage_db(1, 1) ≈ -12.79` dB (tolerance 0.15 dB); and the two compensation-gain values.

Problem 5 — The DFT grid, coherent sampling, and leakage

`dft_bin_frequencies(N, fs)` — signed physical frequency of each of the `N` bins: $k f_s/N$ for $k \leq N/2$, $(k-N) f_s/N$ for $k > N/2$, because an `N`-point DFT samples one w_s -wide period of the sampled spectrum and the upper bins are wrapped negative frequencies.

`coherent_N(f, fs, Nmax)` — the smallest record length that places `f` exactly on a bin, i.e. the smallest `N` making $f*N/fs$ an integer, or `None` if that exceeds `Nmax`. Do it exactly with `Fractions.Fraction` — write f/fs in lowest terms and read off the denominator — rather than searching over floats.

`leakage_fraction(f, fs, N)` — sample $\cos(2\pi f t)$ for `N` points, take $|\text{rfft}|^2$, and return $1 - \text{max_bin}/\text{total}$, the fraction of energy that misses the peak bin. A coherent record returns essentially 0 (the periodic extension is continuous); an incoherent one smears energy across every bin.

This is the practical half of the DFT story from the lecture: the bins are exact samples of the spectrum only when the capture is coherent, and finite captures are never strictly band-limited.

Tested with: the eight bin frequencies for $N = 8$, $f_s = 800$; `coherent_N(250, 1000, 4096) == 4`, `coherent_N(253, 1000, 4096) == 1000`, `coherent_N(253, 1000, 100)` is `None`; and leakage below $1e-9$ at 250 Hz versus above 0.05 at 253 Hz, both with $N = 64$.

Problem 6 — Copy geometry and bandpass (sub-Nyquist) sampling

`spectral_copies(wM, ws, K)` — return `(copies, aliased)`, where `copies` lists the `(low, high)` support of the copy centred at $k \cdot w_s$ for $k = -K \dots K$ in that order, and `aliased` is `True` exactly when neighbouring copies overlap. Derive the overlap condition from the edges rather than quoting it.

`valid_bandpass_rates(f1, f2)` — the interesting generalisation. A signal occupying only `[f1, f2]` need **not** be sampled above $2 \cdot f_2$; it is enough that the copies of the positive and negative bands interleave without colliding. That gives a family of windows

$$2 \cdot f_2 / (m+1) \leq f_s \leq 2 \cdot f_1 / m, \quad m = 1 \dots \text{floor}(f_1/B), \quad B = f_2 - f_1$$

Return the non-empty windows as a list of `(low, high)` pairs sorted ascending. Do not include the unbounded ordinary region $f_s \geq 2 \cdot f_2$.

`bandpass_alias_free(f1, f2, fs)` — `True` when f_s lies inside one of those windows, or is at or above $2 \cdot f_2$. Keep the window edges inclusive against float error.

Tested with: the copies and overlap flags for `(wM, ws) = (2, 10)` and `(6, 10)`; exactly two windows for the band `[100, 140]` Hz, namely `(280/3, 100)` and `(140, 200)`; and the verdicts `True, False, True` for $f_s = 150, 120, 300$.

Problem 7 — Periodic signals: series, DFT bins, harmonic folding

`synthesize(coeffs, N)` — one period of $x[n] = \sum_k a_k \exp(j2\pi kn/N)$ for $n = 0 \dots N-1$, where `coeffs` maps a possibly negative harmonic index k to a possibly complex a_k . The result is complex in general.

`dft_from_series(coeffs, N)` — the predicted DFT, $X[k] = N \cdot a_k$, as a length N complex array. Negative harmonics sit at the wrapped bin $k \% N$. Harmonics that collide after wrapping must **accumulate**, not overwrite — that is exactly how aliasing shows up in the bins.

`recover_series(x, kset)` — invert it: return `{k: X[k % N]/N}` over the requested indices. Valid only when the signal is band-limited to the Nyquist band.

`aliased_series(coeffs, N)` — what the harmonics become when only N samples per period are taken. Fold each k into `(-N/2, N/2]` and **sum** the coefficients that land on the same folded index; that sum is the irreversible part of aliasing. Return a dict keyed by folded index.

Tested with: `coeffs = {0: 1.0, 1: 0.5, -1: 0.5, 3: 0.25j}` at $N = 16$ — `dft_from_series` must match `np.fft.fft(synthesize(...))` to $1e-9$, and `recover_series` must return the original coefficients — and `aliased_series({1: 1.0, 9: 2.0, 5: 1.0}, 8)` giving `3.0` at index `1` and `1.0` at index `-3`.

Problem 8 — Decimation, with and without the anti-alias filter

`decimation_alias(f, fs, M)` — the apparent frequency after keeping every M -th sample with no filtering. Keeping every M -th sample is sampling at f_s/M , so reuse Problem 1 at the reduced rate.

`lowpass_fft(x, fs, fc)` — a brick-wall low-pass done by zeroing every rfft bin above fc . Bin i sits at $i \cdot f_s / \text{len}(x)$ — the length of the time record, not of the rfft output — and `irfft` must be given `n=len(x)` so the output length is preserved for odd lengths too.

`decimate(x, fs, M, antialias=True)` — keep every M -th sample, filtering to the new Nyquist frequency $f_s/(2M)$ first when `antialias` is true.

Then convince yourself of the point of the exercise: a 300 Hz tone sampled at 1000 Hz and decimated by 4 either **vanishes** (filtered) or **reappears at 50 Hz** (not filtered), and in the second case nothing downstream can tell it from a real 50 Hz tone.

Tested with: both alias values; a decimated length of 100 from 400 samples; the unfiltered case identified as a 50 Hz tone at $f_s = 250$ (reusing `identify_tone`); the filtered case below $1e-6$ everywhere; and a legitimate 50 Hz tone surviving the filtered path with amplitude above 0.9.

Problem 9 — Upsampling: zero stuffing, images, interpolation

`zero_stuff(x, L)` — insert $L-1$ zeros after every sample. Note that this changes nothing about the spectrum's content; it only re-reads it on an axis L times wider, so the old copies at multiples of f_s now appear as **images** inside the new baseband.

`interpolate_fft(x, L)` — ideal band-limited interpolation by L , done in the DFT domain: take the rfft, place it at the bottom of a spectrum L times longer, leave the new high bins at zero (that zero region is the ideal low-pass that removes the images), inverse transform at length $N*L$, and scale by L to undo numpy's normalisation. For a signal periodic in the record this reproduces the underlying continuous signal to machine precision — which is a nice concrete demonstration that the sampling theorem is exact, not approximate.

`image_frequencies(f, fs, L, fmax)` — the image frequencies of a tone f after zero stuffing, i.e. the family $k*f_s \pm f$ lying in $(0, fmax]$, sorted, with f itself excluded and no duplicates.

Tested with: `len(zero_stuff(x, 4)) == 256` for a 64-sample input with `z[1] == 0`; interpolation of a 4-cycle cosine by 4 matching the true cosine to $1e-9$; and `image_frequencies(300, 1000, 4, 2000) == [700, 1300, 1700]`.

Problem 10 — A real converter: quantisation noise and aperture jitter

`quantize(x, bits, vref)` — a uniform mid-tread quantiser over $[-vref, vref]$ with 2^{bits} levels. Step size is $2*vref/2^{bits}$; round to the nearest multiple of the step, then clip to the representable range so that an input at $+vref$ cannot produce a code that does not exist.

`snr_db(clean, noisy)` — $10*\log_{10}(\text{signal power} / \text{error power})$ with `error = noisy - clean`.

`theoretical_snr_db(bits)` — the textbook full-scale sine figure $6.02*bits + 1.76$. Then explain to yourself why your **measured** SNR lands a couple of dB below it: the noise is fixed by the step size, but the signal does not fill the input range.

`jitter_samples(f, fs, N, sigma, seed)` — sample $\cos(2*\pi*f*t)$ at jittered times $t_n = n/fs + e_n$, with e_n Gaussian of standard deviation `sigma` drawn from `np.random.default_rng(seed)` so the result is reproducible.

`jitter_rms_bound(f, sigma)` — the predicted rms sampling error. Derive it: for small e the error is $x'(t)*e = -2\pi f \cdot \sin(2\pi f t) \cdot e$, the two factors are independent, $\sin$ contributes an rms of $1/\sqrt{2}$ and e contributes `sigma`. Note that it grows linearly with f — which is why a fast converter needs a low-jitter clock far more than a slow one does.

Tested with: `theoretical_snr_db(10) ≈ 61.96`; a measured SNR between 55 dB and the theoretical value for a 50 Hz tone quantised at 10 bits with `vref = 1.05`; a measured jitter rms within 15% of the bound for $f = 50$, `sigma = 1e-4`; and `quantize([0.0], 8, 1.0) == 0`.

Problem 11 — The moving-average anti-alias filter

`ma_magnitude(w, L)` — the magnitude response of an L -point moving average for w in rad/sample. Summing L samples is a geometric series, so the answer is the Dirichlet kernel $|\sin(wL/2) / (L \cdot \sin(w/2))|$. Both parts vanish at $w = 0$; the limit is 1, so handle that case without dividing. Accept scalars or arrays.

`ma_nulls(L, fs)` — the physical frequencies at which that response is exactly zero, i.e. $k \cdot fs/L$ for $k = 1 \dots L-1$.

`decimate_ma(x, M)` — decimate by M after an M -point moving average (`np.convolve` with `mode="same"`, then take every M -th sample).

Then answer for yourself: why is a length- M box the *cheapest sensible* anti-alias filter for decimation by M ? Look at where its nulls land relative to the frequencies that would fold onto dc, and at how bad its worst-case stopband attenuation really is.

Tested with: unit gain at dc and an exact null at $w = \pi/2$ for $L = 4$; the null list `[250, 500, 750]` for $L = 4$, $f_s = 1000$; and a 300 Hz tone at $f_s = 1000$ decimated by 4 down to amplitude below 0.4 instead of the full 1.0 it would fold to unfiltered.

Problem 12 — Sample-and-hold aperture effect

A real track-and-hold does not read an instant — it averages the input over an aperture of width `tau`.

`sh_magnitude(f, tau)` — the amplitude left at frequency `f` by that averaging: `|sinc(f·tau)|`. Note this is the *same* sinc as the ZOH but set by the aperture, not by the sample period — a different physical mechanism with the same algebra.

`sample_with_aperture(f, fs, N, tau, nsub)` — sample `cos(2πft)` with that aperture, numerically: sample `n` is the average of the input over `[n/fs, n/fs + tau]`, approximated with `nsub` midpoint samples inside the window. Broadcast the `N` instants against the `nsub` offsets; no loop.

Verify two predictions against your simulation, not just one: the recovered amplitude should equal `sh_magnitude(f, tau)`, **and** the recovered phase should be advanced by `π·f·tau`, because averaging over a window also delays by half its width.

Tested with: `f = 300`, `fs = 1000`, `N = 100`, `tau = 1e-3`, `nsub = 400` — amplitude within 0.1% of `sinc(0.3)` and phase within `1e-3` of `π·f·tau` (recovered with `identify_tone` from Problem 1).

Problem 13 — Non-uniform samples, solved by least squares

`design_matrix(times, freqs)` — shape `(len(times), 2·len(freqs))`, holding all the cosine columns first and then all the sine columns. Splitting each tone into a cosine and a sine is the trick that makes the unknowns linear: amplitude and phase are not, but the pair `(a, b)` in `a·cos + b·sin` is.

`fit_tones(times, x, freqs)` — solve with `np.linalg.lstsq` and convert each pair back to `(A, phi)` with `A = hypot(a, b)` and `phi = atan2(-b, a)`. Derive that minus sign rather than guessing it. Report phases in `[0, 2π)`, ordered as `freqs`.

The interesting part is what this shows: with **non-uniform** samples the Nyquist rate in its usual form does not apply at all. What matters instead is whether the linear system is well conditioned — enough samples, well spread, and candidate frequencies not too close together.

Tested with: 200 samples at random times in `[0, 1]` of `1.5·cos(2π·3t + 0.4) + 0.7·cos(2π·11t)`; the matrix shape must be `(200, 4)` and both amplitudes and the phase must come back to `1e-6`.

Problem 14 — Sub-bin frequency estimation

`parabolic_interpolate(mag, k)` — through the three magnitudes `mag[k-1]`, `mag[k]`, `mag[k+1]` there is exactly one parabola; return the offset of its vertex from `k`, `delta = 0.5(a - c)/(a - 2b + c)`, normally in `(-0.5, 0.5)`.

`estimate_frequency(x, fs)` — a frequency estimate finer than the `fs/N` bin spacing: apply a Hann window (a rectangular window's leakage is too peaked for the parabola to fit well), take `|rfft|`, zero bin 0, find the peak, refine, and return `(k + delta)·fs/N`. Fall back to `delta = 0` when the peak sits at either end, where no three-point fit exists.

Tested with: `parabolic_interpolate([1, 3, 2], 1) == 1/6`, and a 253.3 Hz tone sampled at `fs = 1000` with `N = 512` recovered to within 0.3 Hz — against a raw bin spacing of about 1.95 Hz.

Problem 15 — Windows, leakage, and equivalent noise bandwidth

`hann(N)` — the **periodic** Hann window `0.5 - 0.5·cos(2πn/N)`. Periodic (divide by `N`), not symmetric (divide by `N-1`); the periodic form is the one that belongs with the DFT.

`far_leakage(f, fs, N, window, guard=2)` — window a tone, take `|rfft|2`, and return the fraction of total energy falling more than `guard` bins from the peak.

`enbw(window)` — equivalent noise bandwidth in bins, `N·sum(w2)/sum(w)2`: how many bins' worth of white noise the window admits into one bin.

This problem is designed to correct a common misconception, so state the conclusion in your own words when you finish: a window does **not** reduce total spread — it widens the main lobe, putting *more* energy in the immediate neighbours — but it cuts the far sidelobes enormously, and it is the far sidelobes that bury a small tone sitting next to a large one. The `enbw` figure is the price paid.

Tested with: `hann(64)[0] == 0`; `enbw` exactly 1 for a rectangular window and exactly 1.5 for Hann; and Hann's far leakage at 253 Hz, `N = 64`, coming in below **one hundredth** of the rectangular window's.

Problem 16 — Measuring the ZOH images in a real staircase

`stair_signal(samples, L)` — the DAC staircase, oversampled `L` points per sample (`np.repeat`), i.e. what a converter actually emits viewed on a grid fine enough to examine its spectrum.

`zoh_image_ratio(f, fs, L, N)` — sample a cosine, build its staircase at rate `L·fs`, take the rfft, and return the amplitude of the bin nearest `fs - f` divided by the bin nearest `f`. Build the frequency axis from the *staircase* length and the rate `L·fs`; a nearest-bin lookup keeps the answer honest when the record is not perfectly coherent.

`zoh_image_ratio_theory(f, fs)` — the prediction, $|\text{sinc}((fs - f)/fs)| / |\text{sinc}(f/fs)|$, since the staircase is the impulse train shaped by H_0 , so every image is the original line scaled by the sinc envelope at that image's frequency.

Measurement and theory should agree to a few percent; the remaining gap is the finite record, and you should be able to say why.

Tested with: `stair_signal([1, 2], 3) == [1,1,1,2,2,2]`; the theory value exactly $1/9$ for `f = 100`, `fs = 1000`; and measurement within 10% of theory for `L = 8`, `N = 256`.

Problem 17 — A chirp folding off the Nyquist wall

`chirp_samples(f0, f1, dur, fs)` — a linear sweep from `f0` to `f1`. The instantaneous frequency is the **derivative** of the phase, so the phase must be the integral: $2\pi(f_0 \cdot t + \frac{1}{2} \cdot \text{rate} \cdot t^2)$ with `rate = (f1 - f0)/dur`. Writing $2\pi \cdot f(t) \cdot t$ instead is the classic error and sweeps at twice the intended rate — make sure you can see why. Return `(t, x)`.

`instantaneous_frequency(f0, f1, dur, t)` — the true sweep, `f0 + (f1-f0)t/dur`.

`apparent_instantaneous(f0, f1, dur, fs, t)` — the fold applied pointwise, vectorised over an array of times.

Then describe the picture: once the sweep crosses `fs/2` the apparent tone turns around and comes back down, bounces again at 0, and so on — the audible folding of an under-sampled sweep.

Tested with: 500 samples for `f0 = 0`, `f1 = 800`, `dur = 0.5`, `fs = 1000`; instantaneous frequency 400 Hz at `t = 0.25`; and apparent frequencies of 400 Hz at `t = 0.25` (not yet folded) and 300 Hz at `t = 0.4375` (folded back from 700).

Problem 18 — Complex (IQ) sampling: wrapping instead of folding

`complex_fold(f, fs)` — the apparent frequency of $\exp(j2\pi ft)$ sampled at `fs`, in `(-fs/2, fs/2]`. A complex exponential has a **one-sided** spectrum, so nothing reflects: frequencies simply wrap modulo `fs` and keep their sign. The shift trick `((f + fs/2) % fs) - fs/2` does it in one line.

`identify_complex_tone(z, fs)` — the signed frequency of a complex tone from the **full** `np.fft.fft`, unwrapping bins above `N/2` to negative indices before converting to hertz. `rfft` would be wrong here: it assumes a real signal and discards exactly the sign this problem is about.

Compare against Problem 1 and state the difference precisely: 700 Hz at `fs = 1000` is `+300` for a real cosine but `-300` here, and that sign is real information an IQ receiver can use.

Tested with: `complex_fold` giving `-300`, `+300`, `+300` for `f = 700`, `300`, `1300` at `fs = 1000`; and a 700 Hz complex tone identified as `-300` (± 2 Hz) from 250 samples.

Problem 19 — Nyquist zones and spectral inversion

`nyquist_zone(f, fs)` — which zone `f` lives in, counting from 1, where the zones are `[0, fs/2)`, `[fs/2, fs)`, `[fs, 3fs/2)`, ...

`zone_inverts(zone)` — `True` when a zone arrives mirrored. Even zones land on the sampled axis reversed, so a rising tone there appears to fall.

`true_frequency(f_app, zone, fs)` — undo the fold given the apparent frequency and the known zone: `(zone-1)·fs/2 + f_app` for odd zones, `zone·fs/2 - f_app` for even ones.

Note what makes this practical rather than academic: sampling alone can never tell you the zone — that has to come from the analogue bandpass filter in front of the converter, which is the actual design idea behind Problem 6.

Tested with: zones 2, 1, 3 for $f = 700, 300, 1200$ at $f_s = 1000$; the inversion flags for zones 2 and 3; and recovery of 700 Hz from (300, zone 2) and 1200 Hz from (200, zone 3).

Problem 20 — How reconstruction error scales with oversampling

`reconstruction_error(f, fs, dur, t, method)` — maximum absolute error of one method ("sinc", "linear" or "zoh") against the exact cosine.

`error_slope(f, osr_list, method)` — for each oversampling ratio in `osr_list`, sample at $f_s = \text{OSR} \cdot 2f$ (so $\text{OSR} = 1$ is exactly the Nyquist rate), measure the error over one period well inside the record, and fit a straight line through $\log(\text{error})$ against $\log(\text{OSR})$ with `np.polyfit`. Return the slope.

Predict both slopes before running it, from the Taylor error of each interpolator: the ZOH is first-order accurate, so the error should halve when the rate doubles (slope ≈ -1), and linear interpolation is second-order, so it should quarter (slope ≈ -2). This is the quantitative version of the rule of thumb "oversample and a cheap reconstruction will do".

Tested with: $f = 5$, `osr_list = [2, 4, 8, 16]`; the ZOH slope must land in $(-1.35, -0.6)$, the linear slope in $(-2.35, -1.5)$, and linear must be steeper.

Problem 21 — Parseval and band power on the DFT grid

`parseval_error(x)` — $|\sum x[n]^2 - \sum |X[k]|^2 / N|$, which should come out at float noise. The $1/N$ is there because numpy puts no normalisation in the forward transform.

`band_power(x, fs, flo, fhi)` — average power in the band $[f_{\text{lo}}, f_{\text{hi}}]$. Use the full `fft` with `np.fft.fftfreq` so that **both** the positive and the negative bins are counted — a real tone splits its power between them — and divide by N^2 to turn the sum into average power.

Getting Parseval to $\sim 1e-13$ is the cheapest possible check that your transform conventions are right, and worth doing before trusting any power figure computed from a DFT. A unit-amplitude cosine inside the band must give exactly 0.5.

Tested with: a 50 Hz unit cosine, $f_s = 1000$, $N = 1000$: Parseval error below $1e-9$, in-band power exactly 0.5, out-of-band power below $1e-12$.

Problem 22 — Truncating the sinc: how many neighbours are enough

`truncated_sinc_reconstruct(samples, fs, t, K)` — Whittaker-Shannon using only the samples within K of the query time. Build the distance matrix $d = t \cdot f_s - n$ in **sample units**, keep `np.sinc(d)` where $|d| \leq K$ and zero elsewhere, and finish with the same matrix-vector product as the full version. No Python loop.

Then measure the point of the exercise. The ideal filter is unrealizable because the sinc has infinite support and decays only as $1/t$, so the error falls off slowly with K — roughly like $1/K$. Compare several K values against the untruncated result and see how expensive "ideal" really is.

Tested with: a 3 Hz cosine at $f_s = 40$, evaluated on $[2, 3]$: the $K = 5$ error must exceed five times the $K = 60$ error, which must in turn exceed the untruncated error, and $K = 500$ must reproduce the full reconstruction to $1e-9$.

Problem 23 — Rational rate conversion by L/M

`resample_rational(x, L, M)` — change the rate by the factor L/M by interpolating by L (Problem 9's exact FFT interpolation) and then decimating by M .

Upsampling must come **first**, and you should be able to say why in one sentence: doing it the other way throws away information the interpolation could have kept, and can alias irreversibly. Note also that a practical resampler substitutes a polyphase FIR for the FFT interpolation; the ordering argument is the same either way. The conversion needs $L \cdot \text{len}(x)/M$ to be a whole number to land on a clean grid.

Tested with: 64 samples of a 4-cycle cosine converted by $L = 3$, $M = 2$ — length 96, matching the true 4-cycle cosine on the new grid to $1e-9$.

Problem 24 — Reading amplitudes off a real spectrum correctly

`real_spectrum_amplitudes(x, fs)` — return `(freqs, amplitudes)` for a real record. A cosine at an interior bin splits its energy between $+k$ and $-k$, so its amplitude is $2|X[k]|/N$. The dc bin has no partner, and for even N neither does the Nyquist bin — both take $|X|/N$ with **no** factor of two.

Doubling those two is one of the most common ways a perfectly good spectrum ends up 6 dB wrong at exactly the edges people care about. Make your implementation handle odd N correctly too, where there is no Nyquist bin at all.

Tested with: $x = 2 + \cos(2\pi \cdot 100 \cdot n / 1000 + 0.3) + 0.5 \cdot \cos(\pi n)$, $N = 100$, $fs = 1000$ — amplitudes must come out exactly 2.0 at dc, 1.0 at 100 Hz, and 0.5 at the Nyquist bin.

Problem 25 — The stroboscopic (wagon-wheel) effect

`wheel_apparent_rate(rot_hz, frame_hz)` — the apparent rotation rate of a wheel filmed at `frame_hz`, signed. Rotation is genuinely a complex phasor — it has a direction — so this is the **complex wrap** of Problem 18, not the real fold: the answer lies in `(-frame_hz/2, frame_hz/2]` and keeps its sign.

`strobe_direction(rot_hz, frame_hz)` — `+1` forwards, `-1` backwards, `0` frozen: the sign of the apparent rate. Zero is the strobe condition — the wheel turns an exact multiple of the frame rate, every frame catches it in the same place, and it appears to stand still no matter how fast it is really going.

Explain in one line why this problem must use `complex_fold` and not `folded_frequency`, and what physical fact that corresponds to.

Tested with: 29 rev/s at 30 fps giving `-1` (backwards), 31 giving `+1`, 60 giving `0` and direction 0.

Notes on pacing and on the traps

Each problem is meant to take 25–30 minutes: roughly 5 minutes to re-derive the relation from the lecture, 15 to write the functions, and 10 to get the edge cases past the harness. If a problem is running long, it is almost always one of these:

1. `f % fs` in place of the fold `|f - fs·round(f/fs)|` (P1, P8).
2. Forgetting the phase flip on the reflected branch (P1, P2).
3. The normalised vs un-normalised sinc in `np.sinc` (P3, P4).
4. Converting a bin index to hertz with `len(rfft(x))` instead of `len(x)` (P1, P5, P8, P16).
5. Letting the dc bin win the `argmax` (P1).
6. Overwriting instead of accumulating when two harmonics wrap onto one bin (P7).
7. `irfft` without `n=len(x)`, which silently changes the length of odd records (P8).
8. Forgetting the factor `L` after zero-padding a spectrum (P9, P23).
9. Using the real fold where the signal is complex, and losing the sign (P18, P25).
10. Doubling the dc or Nyquist bin amplitude (P24).
11. Writing a chirp phase as `2*pi*f(t)*t` instead of integrating the frequency (P17).
12. Using `rfft` on a complex signal (P18).